In [ ]:
```python
import marimo as mo
from z3 import BitVec, And, Not, Solver, sat, unsat , Or , LShR

def getN():
    while True:
        try:
            N = int(input("digite o n: "))
            return N
        except ValueError:
            print("Input errado, digite novamente")

N = getN()

Vars = ['a' , 'b' , 'x' , 'y' , 'z']
```

Exercício 3 — Multiplicação de inteiros como SFOTS

O programa a modelar multiplica a por b usando só somas e deslocamentos de bits:

```
1:x, y, z = a, b, 0
2:while y != 0
3:    if y é par:  x, y = x << 1, y >> 1      # duplica x,
divide y por 2
4:    else:        y, z = y - 1, z + x        # tira 1 a y e
acumula x em z
# no fim: z == a*b
```

Parâmetros do modelo

- `getN()` pede ao utilizador o número de bits n dos vetores (repete o pedido até receber um inteiro válido). O valor fica guardado em `N`.
- `Vars` é a lista das variáveis do programa: `a` e `b` (os inputs, que nunca mudam), `x`, `y` e `z` (as variáveis de trabalho).

Todas as variáveis são `BitVec` de tamanho `N`, por isso as operações são feitas **módulo** 2^n (o overflow nunca dá erro no Z3, o resultado apenas "dá a volta").

In [ ]:
```python
#codigo feito por llm
def declare(i):
    s = {'pc': BitVec(f'pc{i}', 2)}
    for v in Vars:
        s[v] = BitVec(f'{v}{i}', N)
    return s

def keep(atual, prox, vars):
    return And([prox[v] == atual[v] for v in vars])
```

```
#codigo aproveitado da llm(codigo adaptado)
def init(s):
    return And(s['pc'] == 0 , s['x'] == s['a'] , s['y'] == s['b'] , s['x']
>= 0 , s['y'] >= 0 , s['z'] == 0)

def init2(s):
    return And(s['pc'] == 0 , s['x'] == s['a'] , s['y'] == s['b'] , s['x']
>= 0 , s['y'] >= 0 , s['z'] == 0 , s['a'] < 2**(N//2) , s['b'] < 2**(N//2))

def trans(s, t):
    # s é o passo atual e t o passo seguinte
    pc, pc1 = s['pc'], t['pc']
    # local 0: teste do ciclo
    e0a = And(pc == 0, s['y'] != 0, pc1 == 1, keep(s, t, Vars))
    e0b = And(pc == 0, s['y'] == 0, pc1 == 2, keep(s, t, Vars))
    # local 1: se y é par: x << 1 , y >> 1 , z se nao x , y-1 , z+x
    e1a = And(pc == 1, (s['y'] & 1) == 0 , pc1 == 0,
                t['x'] == s['x'] << 1, t['y'] == LShR(s['y'] , 1) , keep(s,
t ,['a','b','z']))
    e1b = And(pc == 1, (s['y'] & 1) == 1 , pc1 == 0, t['y'] == s['y'] -1 ,
t['z'] == s['z'] + s['x']  ,keep(s, t, ['a' , 'b' , 'x']))
    # local 3: estado final (ciclo sobre si próprio, para a animação)
    e2 = And(pc == 2, pc1 == 2,And(keep(s, t, Vars), And(s['z'] == s['a'] *
s['b']) , t['z'] == t['a'] * t['b']))
    return Or(e0a, e0b, e1a, e1b, e2)

def erro(s):
    return And(s['pc'] == 2 , s['z'] != s['a'] * s['b'])
```

Alínea 1 — Construção do SFOTS

Um SFOTS (*Symbolic Finite Order Transition System*) fica definido por quatro ingredientes: o **estado**, o **estado inicial**, a **relação de transição** e o **estado de erro**. As funções da célula acima codificam cada um deles em Z3.

Estado: `declare(i)`

Cria o estado número `i` da execução, como um dicionário com uma variável simbólica por chave:

- `pc` — *program counter*, o local do autómato onde estamos (2 bits, chega para 3 locais);
- `a`, `b`, `x`, `y`, `z` — as variáveis do programa, cada uma com `N` bits.

O sufixo `i` só serve para distinguir os estados de passos diferentes (`x0`, `x1`, `x2`, ...). O dicionário `declare(0)` representa o estado inicial, `declare(1)` o seguinte, etc.

Auxiliar: `keep(atual, prox, vars)`

Diz ao solver que as variáveis da lista `vars` **mantêm o valor** ao passar do estado `atual` para o estado `prox`. Sem isto, o Z3 poderia escolher valores arbitrários para variáveis que o programa não alterou.

Estado inicial: `init(s)` e `init2(s)`

`init(s)` impõe que `s` é um estado inicial do programa:

- `pc == 0` — começamos no teste do ciclo;
- `x == a`, `y == b`, `z == 0` — a atribuição `x, y, z = a, b, 0`;
- `x >= 0` e `y >= 0` — a pré-condição do programa (inputs não negativos). Atenção: em `BitVec` o `>=` é **com sinal**, logo isto equivale a exigir que o bit mais significativo seja 0.

`init2(s)` é igual a `init(s)` mas acrescenta as restrições `a < 2**N//2` e `b < 2**N//2`, que limitam o tamanho dos inputs para garantir que não há overflow (usada na alínea 3).

Transição: `trans(s, t)`

Descreve **todos os passos possíveis** do estado `s` para o estado seguinte `t`, como uma disjunção de cinco casos (um por ramo do autómato):

Caso	Local	Condição	Efeito
`e0a`	0	`y != 0`	vai para o local 1, nada muda (entra no corpo do ciclo)
`e0b`	0	`y == 0`	vai para o local 2, nada muda (sai do ciclo)
`e1a`	1	`y` par	`x` passa a `x << 1`, `y` passa a `y >> 1` (shift lógico, `LShR`), volta ao local 0; `a`, `b`, `z` mantêm-se
`e1b`	1	`y` ímpar	`y` passa a `y - 1`, `z` passa a `z + x`, volta ao local 0; `a`, `b`, `x` mantêm-se
`e2`	2	—	estado final: fica sempre no local 2 e as variáveis não mudam (ciclo sobre si próprio, necessário para que a execução possa continuar indefinidamente)

A paridade de `y` testa-se com `y & 1`: o bit menos significativo é 0 se e só se `y` é par.

Estado de erro: `erro(s)`

Um estado é de erro quando o programa **terminou** (`pc == 2`) mas o resultado está **errado** (`z != a*b`). É assim que o overflow se manifesta neste modelo: como o Z3

trabalha módulo 2^n, um overflow não dispara nenhum sinal, apenas faz com que o produto final seja incorreto.

Alíneas 2, 3 e 4 — Verificação

A célula seguinte define três verificadores e aplica-os ao SFOTS:

1. `induction_always` — indução simples (alínea 2);
2. `kinduction_always` — k-indução (alíneas 2 e 3);
3. `k_lookahead` — prova de terminação com uma variante (alínea 4).

Em todos os casos o método é o mesmo: pedir ao Z3 um **contra-exemplo**. Se o solver responde `unsat` não há contra-exemplo e a propriedade fica provada; se responde `sat`, o modelo devolvido é um contra-exemplo e a verificação falha.

In [ ]:
```python
# codigo aproveitado da aula
def induction_always(declare,init,trans,inv):
    s = declare(0)
    s_next = declare(1)
    solver=Solver()
    base = And(init(s), Not(inv(s)))
    solver.push()
    solver.add(base)
    if solver.check() == sat:
        print("Falhou o caso base: ", solver.model())
        return False

    solver.pop()
    stop = And(inv(s), trans(s, s_next), Not(inv(s_next)))
    solver.add(stop)
    if solver.check() == sat:
        print("Falhou o passo indutivo: ", solver.get_model())
        return False
    print("invariante valido")
    return True

inv = lambda s: s['x'] * s['y'] + s['z'] == s['a'] * s['b']
induction_always(declare,init,trans,inv)

# codigo aproveitado das aulas
def kinduction_always(declare,init,trans,inv,k):
    solver = Solver()
    states = [declare(i) for i in range(k)]
    base = [init(states[0])]
    for i in range(k-1):
        base.append(trans(states[i], states[i+1]))

    caso_base = (And(base), Or([Not(inv(s)) for s in states]))
    solver.push()
    solver.add(caso_base)
    if solver.check() == sat:
        print("Caso base falhado: ", solver.model())
```

```
        return False
    solver.pop()
    states = [declare(i) for i in range(k+1)]
    inductive = []
    for i in range(k):
        inductive.append(inv(states[i]))
        inductive.append(trans(states[i], states[i+1]))
    inductive.append(Not(inv(states[k])))
    step_case = And(inductive)

    solver.add(step_case)
    if solver.check() == sat:
        print("passo indutivo falhado: ", solver.model())
        return False

    print("invariante valido")
    return True

inv2_forte = lambda s: And(
    Not(erro(s)),
    s['z'] + s['x'] * s['y'] == s['a'] * s['b']
)
kinduction_always(declare,init2,trans,inv2_forte,10)

#codigo aproveitado das aulas
def var_simples(state):
    return state['y']

def k_lookahead(declare,init,trans,V,k):
    s = Solver()
    states = [declare(i) for i in range(k+1)]

    s.add(init(states[0]))

    transitions = [trans(states[i], states[i+1]) for i in range(k)]
    chain = And(transitions)

    decreases = Or(
        V(states[-1]) < V(states[0]),
        V(states[-1]) == 0
        )
    s.add(And(chain, Not(decreases)))

    if s.check() == sat:
        print("Failed: ", s.get_model())
        return False
    print("Variante valida")
    return True

k_lookahead(declare,init,trans,var_simples,2)
```

`induction_always(declare, init, trans, inv)` — indução simples (alínea 2)

Verifica se `inv` é um **invariante** do SFOTS, ou seja, se é verdadeiro em todos os estados alcançáveis. São feitos dois testes:

- **Caso base:** procura um estado inicial que **não** satisfaça `inv` (`init(s) ∧ ¬inv(s)`). Se não existe, o invariante vale à partida.
- **Passo indutivo:** procura um par de estados `s → s_next` em que `inv(s)` é verdadeiro, `trans(s, s_next)` é um passo válido, mas `inv(s_next)` é falso (`inv(s) ∧ trans(s, s') ∧ ¬inv(s')`). Se não existe, o invariante é preservado por cada transição.

Se os dois testes forem `unsat`, imprime `invariante valido`. Caso contrário imprime o contra-exemplo e devolve `False`.

O invariante testado, `inv`, é $x \cdot y + z = a \cdot b$: a cada iteração do ciclo, o que falta multiplicar ($x \cdot y$) mais o que já foi acumulado (z) é sempre o produto pretendido.

`kinduction_always(declare, init, trans, inv, k)` — k-indução (alíneas 2 e 3)

Generalização da indução simples, que olha para k **estados consecutivos** em vez de um só. É útil quando o invariante não é indutivo sozinho (o passo falha por causa de estados inalcançáveis) mas passa a ser quando se assume que se manteve durante k passos.

- **Caso base:** constrói uma execução `s0 → s1 → … → s(k-1)` a partir do estado inicial e procura um estado dessa execução onde `inv` seja falso.
- **Passo indutivo:** constrói `k+1` estados ligados por `trans`, assume que `inv` vale nos primeiros `k` e procura um contra-exemplo em que falha no último.

Aqui é aplicada com `init2` (inputs limitados, alínea 3) e com `k = 10` à propriedade `inv2_forte`. A ideia de `inv2` (definido acima) é reforçar o invariante de multiplicação com a segurança: `¬erro(s)` (o estado de erro nunca é atingido) e $z + x \cdot y = a \cdot b$. Se `kinduction_always` terminar com `invariante valido`, provou-se que o estado de erro é inacessível.

`var_simples(state)` e `k_lookahead(declare, init, trans, V, k)` — terminação (alínea 4)

A propriedade de animação "o programa termina" prova-se com uma **variante** `V`: uma expressão sobre o estado que, de tempos a tempos, tem de diminuir até chegar a 0 (e portanto não pode diminuir para sempre).

- `var_simples` escolhe como variante o valor de `y`. No ciclo, `y` ou é dividido por 2 ou diminui de 1, logo vai descendo até 0.

- `k_lookahead` constrói uma execução de `k` passos a partir do estado inicial e procura um contra-exemplo em que, ao fim desses `k` passos, `y` **não** diminuiu em relação ao início e também **não** é 0.

Com `k = 2`: uma iteração completa do ciclo demora 2 transições (local 0 → 1 → 0), e nela `y` diminui. Se o solver não encontrar contra-exemplo, imprime `Variante valida`, o que mostra que o programa progride sempre em direção ao fim.

Utilização de LLMs neste trabalho

Foram usadas duas conversas com o Claude. Cada uma teve um papel diferente: a primeira serviu para **perceber o exercício e construir o modelo**, a segunda para **melhorar a documentação** (os textos *markdown* deste caderno).

Conversa 1 — compreensão do exercício e base do modelo

Ligação: https://claude.ai/share/90e28273-3836-42f1-b234-76e5afbe241b

- **Objetivo:** perceber como se traduz o programa da multiplicação (ciclo com `if` par/ímpar) para um SFOTS: que é o estado, o que é o estado inicial, como se descreve a transição e o que significa "estado de erro" quando o Z3 trabalha módulo 2^n.
- **O que contribuiu para o código:** deu a base das funções `declare`, `keep`, `init` e `trans` (a organização do estado como dicionário com `pc` e as variáveis, o uso de `keep` para dizer o que não muda, e a divisão da transição em casos, um por ramo do autómato).
- **O que foi adaptado:** `declare` e `keep` foram aproveitadas tal como surgiram; `init` foi adaptada ao enunciado e `init2` foi acrescentada (limita `a` e `b` para evitar overflow, alínea 3); `trans`, `erro` e o invariante `inv2_forte` foram ajustados ao modelo deste exercício. Os comentários `#codigo feito por llm` e `#codigo aproveitado da llm(codigo adaptado)` na célula do modelo marcam estas partes.
- **O que não veio desta conversa:** os verificadores `induction_always`, `kinduction_always` e `k_lookahead` foram aproveitados do material das aulas (ver os comentários `# codigo aproveitado da aula`) e aplicados ao modelo.

Conversa 2 — melhoria dos textos explicativos

Ligação: https://claude.ai/share/7a906a9a-f2f0-498b-9122-013b51aa05b6

- **Objetivo:** tornar mais claras as explicações em *markdown* que acompanham o código, tanto as do SFOTS (alínea 1) como as dos verificadores (alíneas 2, 3 e 4).

- **O que contribuiu:** a reformulação desses textos (por exemplo a tabela dos cinco casos de `trans` e a explicação passo a passo de cada verificador). **Não alterou o código**, apenas a forma como é explicado.
- **Como foi usada:** o texto gerado foi relido e confrontado com o código para confirmar que descreve o que as funções realmente fazem.